

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- **Capture video frames continuously.**
- **Detect multiple faces.**
- **Recognize registered persons using a trained dataset.**
- **Display bounding boxes and identity labels.**

Constraints:

- **Must operate in real time.**
- **Processing delay should be minimized.**
- **Multiple faces must be handled simultaneously.**

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

BEGIN

1. Import OpenCV and required libraries.

2. Load face detection model.

3. Load trained face recognition model
and registered face dataset.

4. Open the live camera.

5. IF camera is not available THEN

 Display "Camera not found"

 STOP

END IF

6. WHILE camera is running DO

 Capture a frame from the camera.

 IF frame is empty THEN

BREAK

END IF

Resize the frame for faster processing.

Convert frame to grayscale.

Detect all faces in the frame.

FOR each detected face DO

 Get face coordinates:

 x, y, width, height.

 Extract the face region.

 Resize and normalize the face.

 Extract facial features.

 Compare features with

 registered face features.

 IF matching score is above threshold THEN

 Set identity = registered person's name.

 ELSE

 Set identity = "Unknown".

 END IF

 Draw bounding box around face.

 Display identity label.

END FOR

Display the processed frame.

IF user presses 'q' or ESC THEN

 BREAK

END IF

END WHILE

7. Release the camera.

8. Close all OpenCV windows.

END

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- **Detect vehicles entering a predefined region of interest.**
- **Track multiple vehicles across consecutive frames.**
- **Count vehicles crossing a virtual line.**
- **Display bounding boxes, tracking IDs, and vehicle count.**

Constraints:

- **Avoid double counting.**
- **Handle multiple vehicles simultaneously.**
- **Maintain near real-time performance.**

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

BEGIN

1. Import OpenCV and required libraries.
2. Open live traffic video.
3. Load vehicle detection model.
4. Initialize multi-object tracker.
5. Define Region of Interest (ROI).
6. Define virtual counting line.
7. Set vehicle_count = 0.
8. Create a record to store:

Vehicle ID

Position

Previous position

Counting status

9. WHILE video is running DO

 Read the current frame.

 IF frame is empty THEN

 BREAK

 END IF

 Resize and preprocess the frame.

 Detect vehicles in the frame.

 FOR each detected vehicle DO

 Identify vehicle type.

 Get bounding box coordinates.

 Send bounding box to tracker.

 END FOR

 Update the tracker.

 FOR each tracked vehicle DO

 Get Tracking ID.

 Get current center position.

 Compare current position

 with previous position.

 IF vehicle crosses the virtual line THEN

 IF vehicle is not already counted THEN

 vehicle_count = vehicle_count + 1.

 Mark vehicle as counted.

 END IF

 END IF

 Update previous position.

 Draw bounding box.

 Display Tracking ID.

END FOR

Draw ROI and counting line.

Display vehicle_count on screen.

IF user presses 'q' or ESC THEN

BREAK

END IF

END WHILE

10. Release video source.

11. Close all OpenCV windows.

END